

SideChannel

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

There's something fishy about this PIN-code checker, can you figure out the PIN and get the flag?

Download the PIN checker program here: [pin_checker](#)

Additional details will be available after launching your challenge instance.

Hints

- Read about "timing-based side-channel attacks."
 - Attempting to reverse-engineer or exploit the binary won't help you; you can figure out the PIN just by interacting with it and measuring certain properties about it.
 - Don't run your attacks against the master server; it is secured against them. The PIN code you get from the [pin_checker](#) binary is the same as the one for the master server.
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `xxd` – View hexadecimal representation
 - `chmod` – Make binary executable
 - `time` – Measure execution time
 - **Python** – Automated timing attack script
 - **Bash** – Alternative timing attack script
 - `nc` (netcat) – Connect to remote service
-

Complete Solution

Step 1: Download and Prepare

Download the file `pin_checker` from the challenge page.

Once you've figured out the PIN (and gotten the checker program to accept it), connect to the master server using the provided netcat endpoint and provide the PIN to get your flag.

Step 2: Basic File Inspection

First, verify the file type:

```
file pin_checker
```

Expected output:

```
pin_checker: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamically  
linked, interpreter /lib/ld-linux.so.2, stripped
```

The binary is a **32-bit ELF executable** (stripped – no symbols).

Step 3: Search for Visible Text

Check if the flag appears as plain text:

```
strings pin_checker | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly embedded in the binary.

Step 4: Inspect Metadata

```
exiftool pin_checker
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : pin_checker
Directory                    : .
File Size                    : 8.6 MB
...
File Type                    : ELF executable
CPU Architecture             : 32 bit
CPU Byte Order               : Little endian
Object File Type             : Executable file
CPU Type                     : i386
```

Nothing suspicious in the metadata.

Step 5: Hexdump Analysis

```
xxd pin_checker | head
```

Output:

```
00000000: 7f45 4c46 0101 0100 0000 0000 0000 0000  .ELF.....
00000010: 0200 0300 0100 0000 9c90 0408 3400 0000  .....4...
00000020: f4f8 8200 0000 0000 3400 2000 0800 2800  .....4. ...(
...
```

The `7F 45 4C 46` (`ELF`) magic bytes confirm this is an ELF binary.

Step 6: Run the Binary

Make it executable:

```
chmod +x pin_checker
```

Run it:

```
./pin_checker
```

Output:

```
Please enter your 8-digit PIN code:
12345678
8
Checking PIN...
Access denied.
```

The binary expects an **8-digit PIN**. Entering the wrong PIN results in "Access denied."

Step 7: Understand the Timing Side-Channel

The hint mentions "**timing-based side-channel attacks**". This means the binary takes slightly different amounts of time to validate the PIN depending on how many leading digits are correct.

How it works:

1. The program likely checks the PIN **one digit at a time**
2. If the first digit is wrong, it rejects immediately (fast)
3. If the first digit is correct, it proceeds to check the second digit (slower)
4. Each correct digit adds a measurable delay

By measuring the execution time for different inputs, we can **brute-force the PIN digit by digit** – a classic side-channel attack.

Step 8: Measure Execution Time (Manual Test)

Test the timing difference between a completely wrong PIN and a partially correct one:

```
time echo "00000000" | ./pin_checker
```

```
time echo "12345678" | ./pin_checker
```

You should notice a slight difference in execution time when the first digit matches.

Step 9: Python Automated Attack Script

python

```
#!/usr/bin/env python3
import time
import subprocess

def get_execution_time(pin_input):
    """Measure how long the pin_checker takes to validate a PIN."""
    start = time.time()
    process = subprocess.Popen(
        ['./pin_checker'],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE
    )
    process.communicate(input=pin_input.encode())
    end = time.time()
    return end - start

def find_pin():
    pin = ""

    # Iterate through each of the 8 positions
    for i in range(8):
        max_time = 0
        best_digit = '0'

        # Test each digit 0-9 for the current position
        for digit in '0123456789':
            # Construct test PIN: known prefix + test digit + zeros
            test_pin = pin + digit + '0' * (7 - i)

            execution_time = get_execution_time(test_pin)

            # The digit that takes longest is likely correct (more checks performed)
            if execution_time > max_time:
                max_time = execution_time
                best_digit = digit

        pin += best_digit
        print(f"Found digit {best_digit} → Current PIN: {pin}")

    return pin

if __name__ == "__main__":
```

```

print("[*] Starting timing-based side-channel attack...")
pin = find_pin()
print(f"\n[+] Final PIN: {pin}")

# Verify the PIN
print("\n[*] Verifying PIN...")
print(pin)
process = subprocess.Popen(['./pin_checker'], stdin=subprocess.PIPE,
stdout=subprocess.PIPE)
stdout, _ = process.communicate(input=pin.encode())
print(stdout.decode())

```

Step 10: Bash Automated Attack Script

If you prefer a shell script:

```

#!/bin/bash

# Ensure the binary is executable
chmod +x pin_checker

PIN=""
LEN=8

# Measure execution time with multiple samples for accuracy
measure_time() {
    local guess="$1"
    local samples=5
    local total_time=0

    for ((s=0; s<samples; s++)); do
        local time_output=$( { time echo "$guess" | ./pin_checker > /dev/null 2>&1; }
2>&1 | grep real | awk '{print $2}')
        if [[ "$time_output" =~ ([0-9]+)m([0-9.]+)s ]]; then
            local minutes=${BASH_REMATCH[1]}
            local seconds=${BASH_REMATCH[2]}
            local current_time=$(awk "BEGIN {print $minutes * 60 + $seconds}")
        else
            local current_time=0
        fi
    done
}

```

```

        total_time=$(awk "BEGIN {print $total_time + $current_time}")
done

awk "BEGIN {printf \"%.6f\\\", $total_time / $samples}"
}

# Brute-force each digit
for i in $(seq 0 $((LEN - 1))); do
    MAX_TIME=0
    BEST_DIGIT=0

    echo "[*] Testing position $((i+1))/8..."

    for d in $(seq 0 9); do
        REMAINING=$((LEN - i - 1))
        ZEROS=$(printf "%0${REMAINING}d" 0)
        GUESS="${PIN}${d}${ZEROS}"

        AVG_TIME=$(measure_time "$GUESS")

        IS_GREATER=$(awk "BEGIN {print ($AVG_TIME > $MAX_TIME) ? 1 : 0}")

        if [ "$IS_GREATER" -eq 1 ]; then
            MAX_TIME=$AVG_TIME
            BEST_DIGIT=$d
        fi

        echo "    Digit $d: ${AVG_TIME}s"
    done

    PIN="${PIN}${BEST_DIGIT}"
    echo "✅ Position $((i+1)): $BEST_DIGIT"
    echo "    Current PIN: $PIN"
    echo ""
done

echo "===== "
echo "🔒 Final PIN: $PIN"
echo "===== "

# Verify the PIN
echo ""
echo "[*] Final verification..."
echo "$PIN" | ./pin_checker

```

Step 11: Connect to the Master Server

Once you have the correct 8-digit PIN:

```
nc saturn.picoctf.net 50724
```

Output:

```
Verifying that you are a human...  
Please enter the master PIN code:
```

Enter the PIN you discovered.

Output:

```
Password correct. Here's your flag:  
picoCTF{t1m1ng_4tt4ck_<REDACTED>}
```

Note: The port number may change each time you launch the instance.

Step 12: Important Note on Timing Variability

Timing attacks are sensitive to system load. If the PIN isn't found on the first run:

1. **Increase the number of samples** – Use 10-20 samples instead of 5
2. **Reduce system load** – Close other applications
3. **Run multiple times** – The correct PIN will consistently have longer execution times
4. **Run on a dedicated machine** – Virtual machines may have inconsistent timing

Final Result

```
picoCTF{t1m1ng_4tt4ck_<REDACTED>}
```

Note: The actual PIN and flag are redacted here. In the real challenge, running the timing attack script will reveal the correct PIN and flag.

Key Concepts

Concept	Description
Side-Channel Attack	An attack that exploits physical characteristics of a system (timing, power consumption, electromagnetic emissions) rather than cryptographic weaknesses.
Timing Attack	A side-channel attack where the attacker measures how long a system takes to respond to different inputs.
PIN Validation	The program likely checks digits sequentially, rejecting early on the first incorrect digit.
Sequential Comparison	Each correct digit adds a measurable delay, revealing which digits are correct.
Brute-Force with Timing	Instead of trying 10^8 possibilities, we test $8 \times 10 = 80$ possibilities using timing differences.
Signal-to-Noise Ratio	Multiple samples average out system noise to reveal the timing difference.

Pro Tips

1. **Run the attack locally first** – The `pin_checker` binary and the master server use the same PIN. Discover it locally before connecting.
2. **Multiple samples are essential** – System noise can affect timing. Average 5-10 runs per digit.
3. **The timing difference is small but measurable** – Correct digits take a few milliseconds longer. Use high-precision timing.
4. **Don't attack the remote server directly** – The hint explicitly says it's secured against timing attacks. Attack the local binary.
5. **Stripped binaries don't matter** – Side-channel attacks don't require reverse engineering; they only need to measure the program's behavior.

6. **Pin is the same for all instances** – Once you find the PIN locally, it works for the remote server.
-

Alternative Python One-Liner (Simpler)

A more compact version without classes:

```
import time, subprocesspython  
  
def time_pin(pin):  
    start = time.time()  
    subprocess.run(['./pin_checker'], input=pin.encode(), capture_output=True)  
    return time.time() - start  
  
pin = ''  
for i in range(8):  
    pin += max('0123456789', key=lambda d: time_pin(pin + d + '0'*(7-i)))  
    print(f'PIN: {pin}')
```

Conclusion

The `pin_checker` binary validates PINs sequentially, checking one digit at a time. By measuring the execution time for different inputs, we can determine which digits are correct – correct digits cause the program to spend more time checking further digits.

This is a **timing-based side-channel attack**. Instead of brute-forcing 100 million possibilities (10^8), we only need to test $8 \text{ positions} \times 10 \text{ digits} = 80 \text{ combinations}$.

Once we discovered the PIN locally, we connected to the remote server, entered the PIN, and received the flag.

This challenge demonstrates:

- **Side-channel attacks** – Using timing differences to extract secrets
- **Sequential validation vulnerabilities** – Early exit on incorrect input leaks information
- **Practical cryptanalysis** – No reverse engineering or binary exploitation required

Key takeaway: Even if a system is cryptographically secure, its **implementation** can leak information through side channels like timing. Always validate inputs in a constant-time manner to

prevent such attacks.